UNIVERSIDAD NACIONAL DE UCAYALI

FACULTAD DE INGENIERÍA DE SISTEMAS E INGENIERÍA CIVIL

ESCUELA DE PROFESIONAL DE INGENIERÍA DE SISTEMAS

PATRONES DE DISEÑO

PROGRAMACIÓN ORIENTADA A OBJETOS II

Docente : Ing. M.Sc. Edson Greig Cáceres Cárdenas

Pucallpa – Perú

2026

INTRODUCCIÓN

Los patrones de diseño se derivaron de ideas planteadas por Christopher Alexander (Alexander et al., 1977), quien sugirió que había ciertos patrones comunes de diseño de construcción que eran relativamente agradables y efectivos. **El patrón es una descripción del problema y la esencia de su solución**, de modo que la solución puede reutilizarse en diferentes configuraciones. El patrón no es una especificación detallada. Más bien, puede considerarla como una descripción de sabiduría y experiencia acumuladas, una solución bien probada a un problema común.

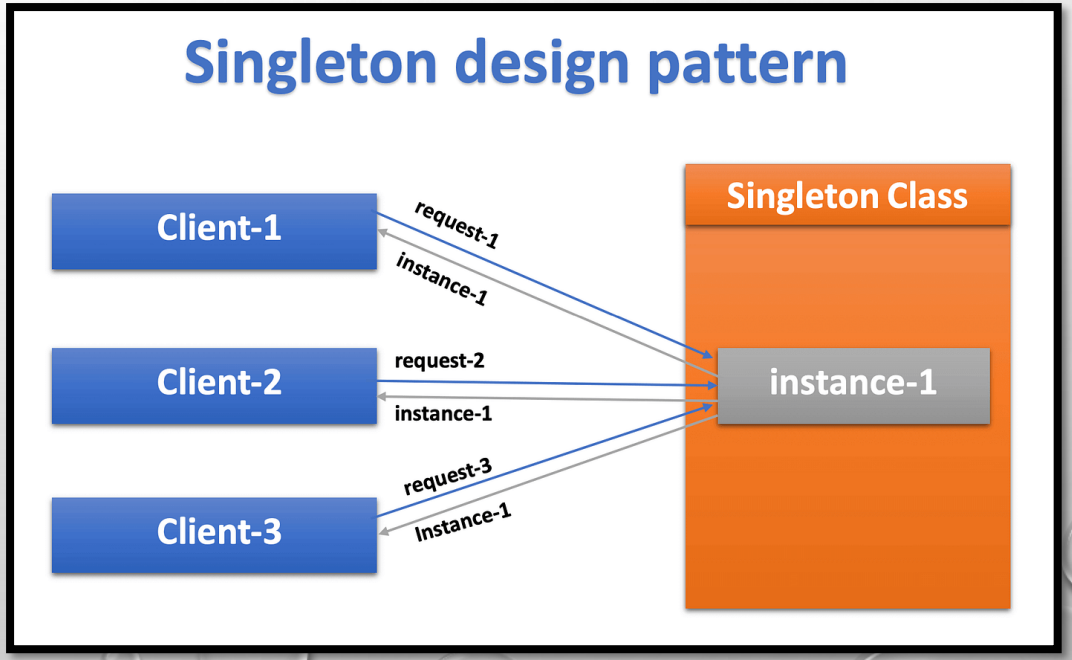
Los patrones causaron un enorme impacto en el diseño de software orientado a objetos. Como son soluciones probadas a problemas comunes, se convirtieron en un vocabulario para hablar sobre un diseño. **Los patrones publicados se suelen apoyar en características de objetos como herencia y polimorfismo para dar generalidad.**

PATRONES DE DISEÑO

Design patterns provide low-level solutions related to the implementation of commonly occurring object-oriented problems. In other words, the design pattern suggests a specific implementation for the specific object-oriented programming problem.

Los patrones de diseño son plantillas de solución para problemas comunes en el desarrollo de software. No son fragmentos de código que copias y pegas, sino conceptos que adaptas a tu caso particular.

- **Beneficios**: Reutilización de código, mantenibilidad, lenguaje común entre desarrolladores y reducción de la deuda técnica.
- **Clasificación**:
  1. Creacionales: Cómo se crean los objetos.
  2. Estructurales: Cómo se componen las clases y objetos.
  3. Comportamiento: Cómo interactúan y se reparten responsabilidades.

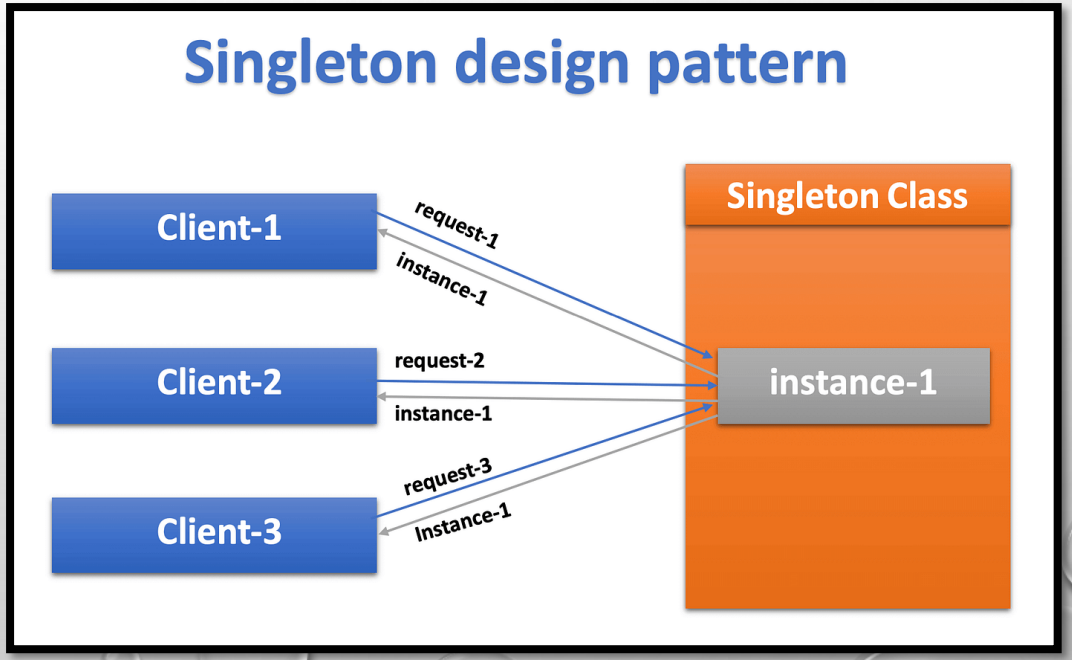


PATRONES DE DISEÑO CREACIONALES

Los **patrones de diseño creacionales** abordan los mecanismos de creación de objetos, buscando crearlos de forma adecuada a la situación. El objetivo principal es permitir que los sistemas sean independientes del proceso de creación de objetos, garantizando al mismo tiempo la flexibilidad y la facilidad de mantenimiento del código.

Hay cinco patrones de creación clave:

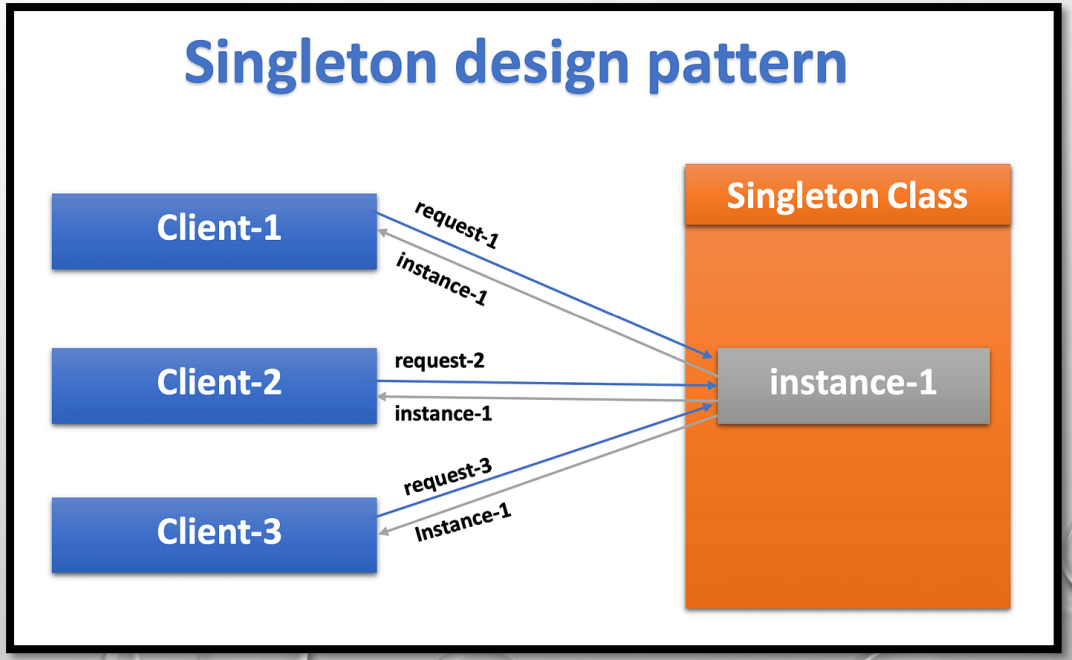
- Patrón Singleton
- Patrón de método de fábrica
- Patrón de fábrica abstracto
- Patrón de constructor
- Patrón de builder



PATRONES DE DISEÑO CREACIONALES

Principios claves:

1. **Desacoplamiento** : Los patrones de diseño creacionales desacoplan la creación de un objeto de su uso. Esto permite que el código del cliente se centre en el uso de los objetos en lugar de en su creación.
2. **Encapsulación** : Los patrones a menudo ocultan la complejidad de la creación de objetos, brindando a los desarrolladores una interfaz fácil de usar para obtener objetos.
3. **Flexibilidad** : al abstraer la lógica de creación, estos patrones permiten un código más flexible que puede adaptarse a los cambios en la lógica de creación de objetos sin afectar al resto del sistema.

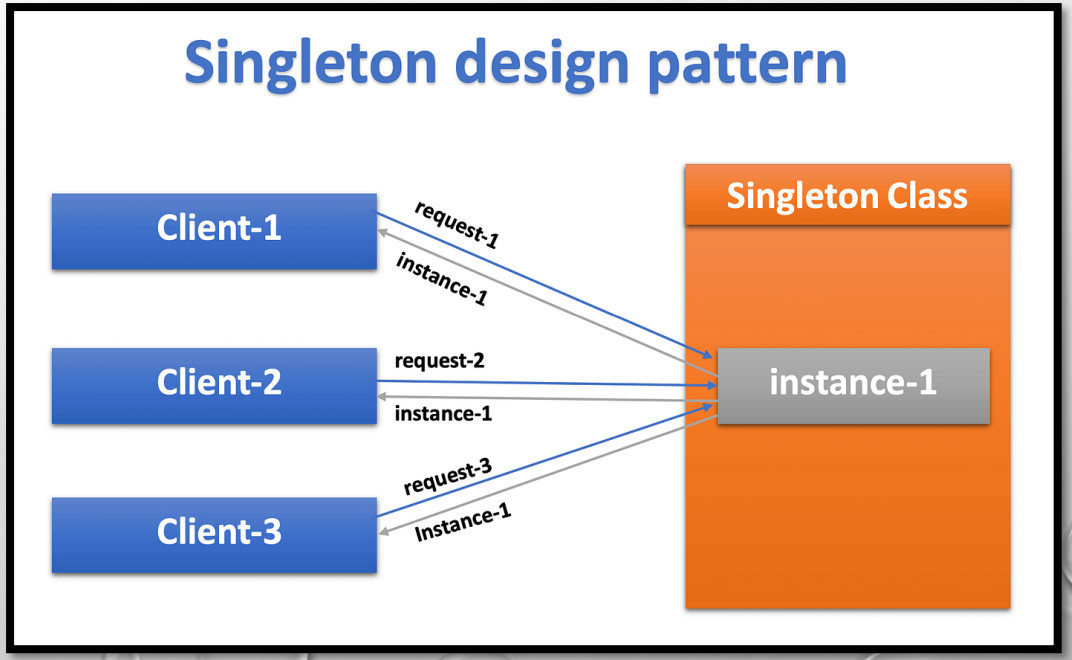


PATRONES DE DISEÑO CREACIONALES

Patron singleton

Java

```java
public class DatabaseConnector {
    private static DatabaseConnector instance;

    // El constructor es privado para evitar instanciación extern
    private DatabaseConnector() { }

    public static DatabaseConnector getInstance() {
        if (instance == null) {
            instance = new DatabaseConnector();
        }
        return instance;
    }
}
```



PATRONES DE DISEÑO CREACIONALES

Patron builder

Permite construir objetos complejos paso a paso. Es ideal cuando un objeto tiene muchos atributos y el constructor se vuelve confuso.

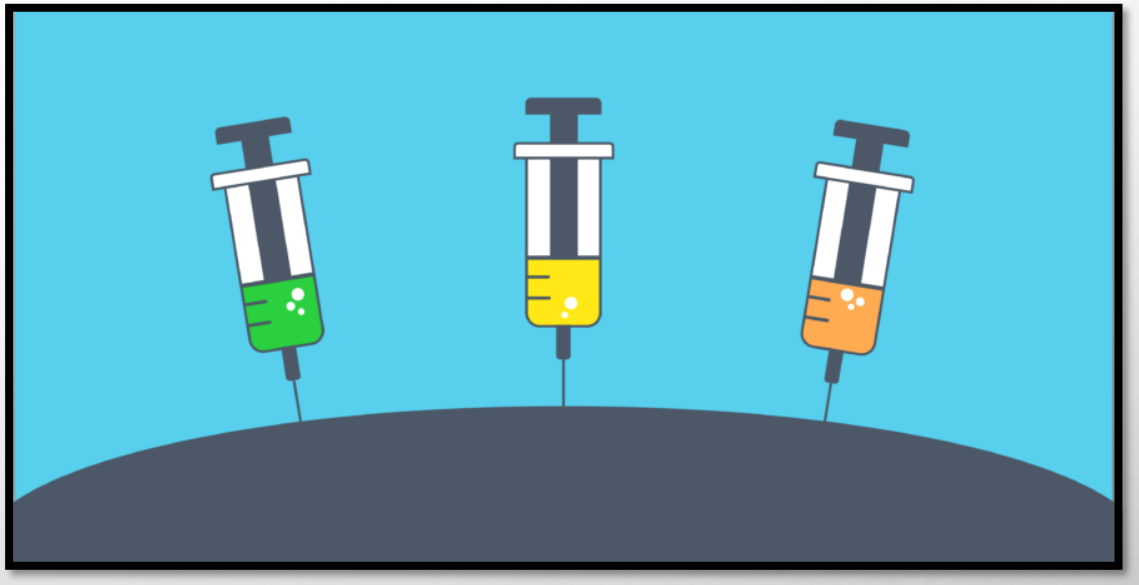
```java
public class Usuario {
    private final String nombre;  // Requerido
    private final String email;   // Requerido
    private final int edad;       // Opcional

    private Usuario(Builder builder) {
        this.nombre = builder.nombre;
        this.email = builder.email;
        this.edad = builder.edad;
    }

    public static class Builder {
        private String nombre;
        private String email;
        private int edad;

        public Builder(String nombre, String email) {
            this.nombre = nombre;
            this.email = email;
        }

        public Builder setEdad(int edad) {
            this.edad = edad;
            return this;
        }

        public Usuario build() {
            return new Usuario(this);
        }
    }
}

// Uso:
Usuario u = new Usuario.Builder("Juan", "juan@mail.com").setEdad(25).build();
```

PATRONES DE DISEÑO ESTRUCTURALES

Tratan sobre la composición de clases y objetos para formar estructuras más grandes y complejas. Estas estructuras permitirán que se agreguen nuevas funcionalidades.

Algunos ejemplos de estos patrones son:

- Composite
- Facade
- Adapter
- Proxy
- Decorator
- Inyección de dependencias



PATRONES DE DISEÑO ESTRUCTURALES

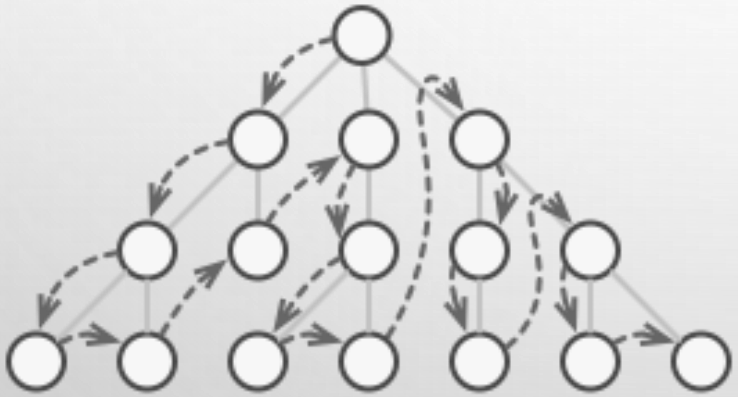
Patron de inyección de dependencias

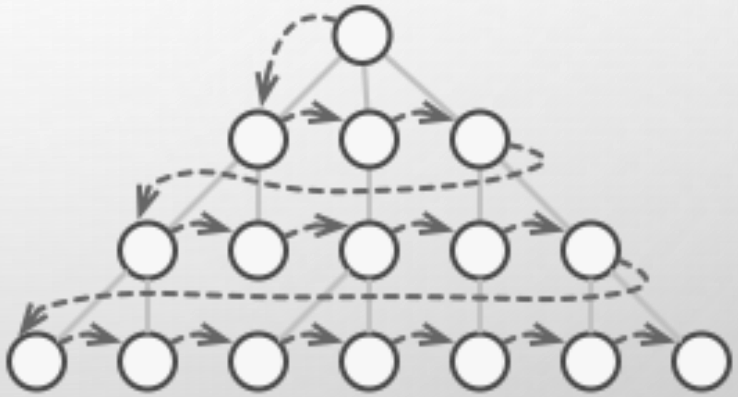
Dependency injection is a programming technique that makes a class independent of its dependencies. It achieves that by decoupling the usage of an object from its creation.

```java
// Interfaz para desacoplar
interface ServicioMensajería {
    void enviar(String mensaje);
}

// Implementación concreta
class EmailServicio implements ServicioMensajería {
    public void enviar(String mensaje) { System.out.println("Enviando Email: " +
}

// Clase que recibe la dependencia (Inyección por Constructor)
class Notificador {
    private final ServicioMensajería servicio;

    public Notificador(ServicioMensajería servicio) {
        this.servicio = servicio;
    }

    public void notificar(String msg) {
        servicio.enviar(msg);
    }
}
```

PATRONES DE DISEÑO DE COMPORTAMIENTO

Mientras que los patrones creacionales se encargan de "quién crea qué", los de **comportamiento** se centran en los algoritmos y la asignación de responsabilidades entre objetos. No solo describen los objetos, sino también los **patrones de comunicación** entre ellos.





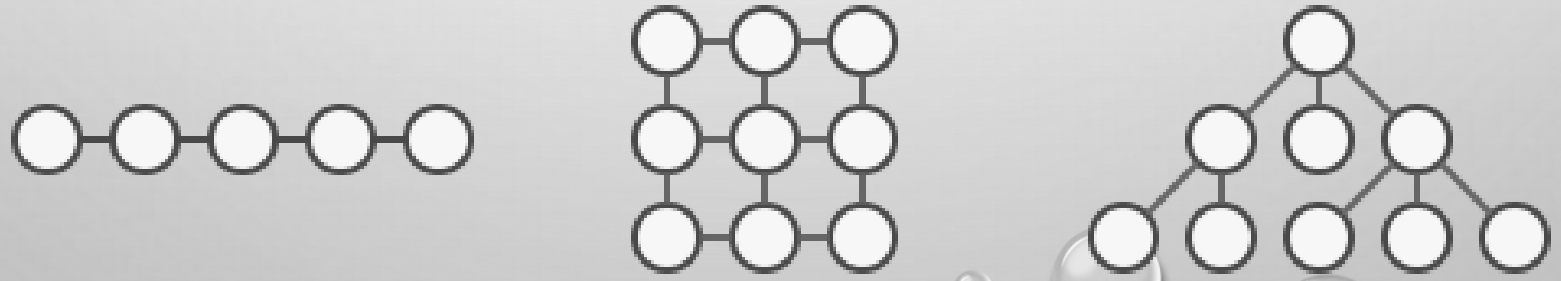
PATRONES DE DISEÑO DE COMPORTAMIENTO

Patron Iterator

¿Cuál es el problema que resuelve?

Imagina que tienes una colección compleja de datos: una lista, un árbol binario o un grafo. Para que alguien más use esos datos, normalmente tendría que conocer la estructura interna (saber si es un array, un nodo con punteros, etc.).

El problema surge cuando quieres recorrer los elementos sin exponer cómo están guardados y sin llenar tu lógica de negocio con ciclos for o while específicos para cada estructura.



PATRONES DE DISEÑO DE COMPORTAMIENTO

Patron Iterator

```java
interface Iterador<T> {
    boolean tieneSiguiente();
    T siguiente();
}
```

```java
import java.util.ArrayList;
import java.util.List;

class ListaTareas {
    private List<String> tareas = new ArrayList<>();

    public void agregarTarea(String t) { tareas.add(t); }

    // Método que devuelve el iterador
    public Iterador<String> crearIterador() {
        return new TareaIterador(this.tareas);
    }
}
```

```java
class TareaIterador implements Iterador<String> {
    private List<String> lista;
    private int posicion = 0;

    public TareaIterador(List<String> lista) {
        this.lista = lista;
    }

    @Override
    public boolean tieneSiguiente() {
        return posicion < lista.size();
    }

    @Override
    public String siguiente() {
        return lista.get(posicion++);
    }
}
```

Conclusiones

1. No son "Recetas", son "Planos"

Los patrones de diseño no se copian y pegan. Son **guías conceptuales** que deben adaptarse al contexto de cada proyecto. Su mayor valor reside en que ofrecen una solución probada a problemas que otros desarrolladores ya enfrentaron y resolvieron de forma eficiente.

2. Crean un "Vocabulario Común"

Esta es quizás la conclusión más práctica para el trabajo en equipo. En lugar de explicar durante 10 minutos cómo vas a estructurar una creación de objetos, simplemente dices: *"Vamos a usar un **Builder**"*. Esto mejora drásticamente la comunicación técnica y reduce la ambigüedad.

3. Facilitan el Mantenimiento y la escalabilidad

Al aplicar patrones, el código tiende a estar más **desacoplado**:

- **Flexibilidad:** Cambiar una parte del sistema no rompe el resto.
- **Extensibilidad:** Es más fácil añadir nuevas funciones.

GRACIAS